

방학 - 4주차

날짜 2026년 8월 15일

생각나는거

- 라즈베리파이
- 속도 개선
- 대시보드
- 배경지식 정리

속도개선

- 지금 피처가 너무 많아서 느려졌음.
- 성능을 유지하면서 빠르게 만들 방법을 고민해야함

최종 피처 (29개 × 3센서 = 87채널)

그룹	컬럼	개수
기본 4특징	peak_freq_bin, band_start, band_end, rms	4
rpm	rpm	1
확장특징 (절대속)	band_ratio_0~7, centroid, entropy, rolloff, log_flatness	12
확장특징 (차수속)	ord_band_ratio_0~7, ord_centroid, ord_entropy, ord_rolloff, ord_log_flatness	12

센서 3개(Current_U, Vib_Motor, Vib_TM)를 early fusion으로 스택 → 29 × 3 = 87채널.

실행 명령은 모터_학습_v3.py 에 --ext --ord .

parquet에는 33컬럼이 있지만 group_key, fault_class, time_idx, temperature 4개는 메타/제외라 학습에 안 들어가.

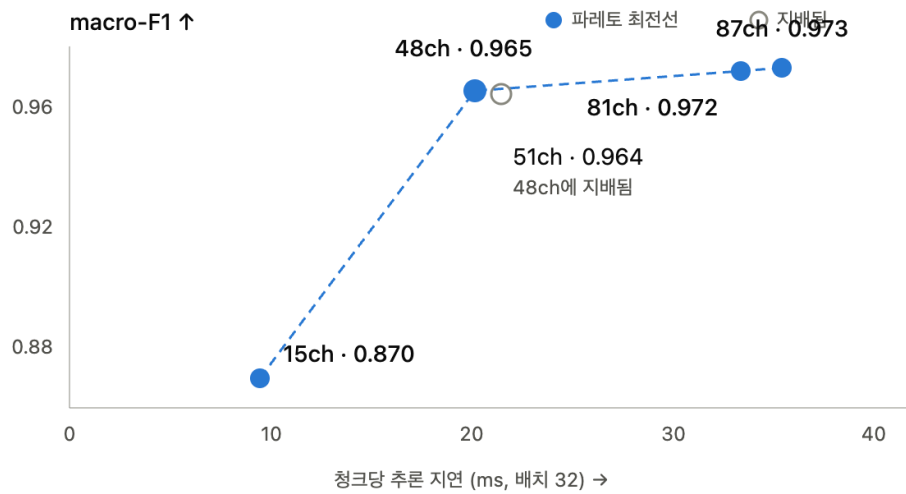
채택 근거 (요약)

- A4 (51ch, 절대속까지): macro-F1 평균 0.964, DEMAG recall 0.971/0.883/0.928
- B (87ch, 차수속 추가): macro-F1 평균 **0.973**, DEMAG recall **0.987/0.937/0.941**
- DEMAG→NORMAL 오분류 704건 → 433건 (−38%)
- 비용: 학습 28.2분 → 38.0분

기각/보류된 것

- temperature — 기각 (A3, 전 차종 하락)
- band_ratio_7 계열 제외안 (81ch, C안) — 미실행. 절대속만 뺀 4-c에선 KONA DEMAG recall +2.0%p로 미세 우위였는데, ord_band_ratio_7 까지 뺀 버전은 아직 안 돌렸어. [차수정규화 실험 결과.md](#) 58에 남은 작업으로 있음.

즉 87채널이 현재 확정본이지만, 81채널 C안 비교가 아직 남아 있어서 “최종”이 완전히 못박힌 상태는 아니야.



구성	채널 구성	macro-F1	지연(ms)	DEMAG 평균
15ch		0.870	9.5	0.781
48ch		0.965	20.2	0.935
51ch 지배됨		0.964	21.5	0.927
81ch		0.972	33.4	0.951
87ch		0.973	35.4	0.955

기본 4특징	peak_freq_bin, band_strat, band_end, rms	4
rpm	rpm	1
확장특징(절대측)	band_ratio_0~7, centroid, entropy, rolloff, log_flatness	12
확장특징(차수측)	ord_band_ratio_0~7, ord_centroid, ord_entropy, ord_rolloff, ord_log_flatness	12

채널수	피쳐	marco-F1	DEMAG recall (IONIQ/KONA/NIRO)	지연(배치32) (ms)
4채널	기본 4특징			
15채널(5*3)	기본 4특징 + RPM	0.870	0.813/0.759/0.770	9.5

48채널(16*3)	기본 4 + RPM + 절대축 11	0.965	0.974/0.903/0.927	20.2
51채널(17*3)	기본 4 + RPM + 절대축 12	0.964	0.971/0.883/0.928	21.5
51채널(17*3)	기본 4 + RPM + 상대축 12	0.9635	0.9225	21.3
81채널(27*3)	기본 4 + RPM + 절대축 11 + 상대축 11	0.972	0.983/0.931/0.940	33.4
87채널(29*3)	기본 4 + RPM + 절대축 12 + 상대축 12	0.973	0.987/0.937/0.941	35.4

새로운 51채널(기본4 + RPM + 상대축12) 돌리는 코드:

```
python 모터_학습_v3.py --data "...\특징데이터" --out "...\모델_ord_only" --ext --ord --drop band_ratio_0,band_ratio_1,band_ratio_2,band_ratio_3,band_ratio_4,band_ratio_5,band_ratio_6,band_ratio_7,centroid,entropy,rolloff,log_flatness
```

학습리포트 fuse_rpm_ext_no_band_ratio_7_no_ord_band_ratio_7.txt

피쳐 선택의 근거 → 가로는 피쳐 세로는 속도 + 정확도 최적점을 찾기

정확도와 속도의 트레이드오프 그래프



커널 축소가 먼저인 이유가 이거야. 정보를 안 버리면서 속도를 얻는 쪽이니까 밀쳐야 본전이고, 여기서 목표를 달성하면 채널을 깎을 필요 자체가 없어져

	채널 축소	커널 축소
뭘 줄이나	입력 정보량	계산 반복 횟수
바꾸는 것	FEATURE_COLS	NUM_KERNELS
정확도 손실	있음 (0.955 → 0.935)	작을 것으로 예상
특징 추출 비용	같이 줄어듦	안 줄어듦

하드웨어랑 비슷하게 측정하는 방법?

1. 커널 수별 성능 가격표 (제일 유용, PC로 지금 가능)

커널 10000 / 5000 / 2000 / 1000으로 각각 학습해서 macro-F1과 DEMAG recall 기록. 4번 돌리면 반나절이야. Pi 도착하는 순간 측정값 보고 최적점을 바로 고를 수 있고, 이 표 자체가 ****"성능-속도 트레이드오프 곡선"*****이라는 발표 자료가 돼. 경량화 프로젝트에서 이런 곡선은 그 자체로 결과물이야.

2. PC에서 latency 측정 프로토콜 완성

Pi에서 돌릴 스크립트를 미리 PC에서 만들고 검증해둬. 워업 5회 → 100회 반복 → 중앙값/p95, transform과 predict 분리 측정, 메모리 피크 기록까지. 그리고 **PC 기준값을 먼저 확보**해두면 나중에 "PC 대비 Pi가 몇 배"라는 비교표가 자동으로 나와. 총1 검증용 (입력 100개, PC 예측 확률) npz도 지금 만들어두면 돼.

3. 1코어 제한 실험 (Pi 흥내내기)

PC에서 `taskset -c 0` 으로 1코어만 써서 돌려보면 Pi 환경을 거칠게나마 흉내낼 수 있어. 절대값은 못 믿지만 "코어 수에 얼마나 비례하는지"를 알면 Pi 예측 정확도가 올라가.

순서

Pi 4 측정이랑 겹치는데, 커널 스왑을 먼저 돌려. 이유는 Pi에서 뭘 찰지가 이 결과로 정해지거든. 지금 Pi를 재면 5개 구성을 재야 하는데, 커널 스왑 끝나면 후보가 2~3개로 좁혀져서 Pi 측정이 훨씬 가벼워져.

1. 커널 스왑 3회 (하룻밤)
2. 판정 → 후보 2~3개 확정
3. Pi 4에서 그 후보만 측정
4. 최종 채택